Final_Report (WebMiner).pdf

by Muhammad Syahmi Faris RUSLI

Submission date: 25-May-2026 05:52AM (UTC-0700)
Submission ID: 2969134347
File name: Final_Report_WebMiner_.pdf (1.31M)
Word count: 4256
Character count: 22983

UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Department of Computer Science
Faculty of Computing

SECP3133 – HIGH PERFORMANCE DATA PROCESSING

Project 1: Optimizing High Performance Data Processing for Large Scale Web Crawlers

Programme	:	Bachelor of Computer Science *(Data Engineering)*
Subject Code	:	SECP3133-01
Subject Name	:	High Performance Data Processing
Session-Sem	:	2025/2026-02

Prepared by : 1) Muhammad Afiq Danish bin Mohd Hazni (A23CS0118)
2) Muhammad Syahmi Faris bin Rusli (A23CS0138)
3) Ng Yu Hin (A23CS0148)

Group : WebMiner

Section : 01

Lecturer : Prof. Madya. Ts. Dr. Mohd Shahizan bin Othman

Table of Contents

1.0 Introduction

Background of the project

The exponential growth of unstructured data on the web necessitates not only robust extraction methods but also highly efficient processing pipelines. This project focuses on optimizing high-performance data processing for large-scale web crawlers. Extracting data from modern websites introduces significant challenges, such as bypassing dynamic JavaScript rendering. Furthermore, the extracted raw data is often noisy, containing inconsistencies like numbers stored as strings, compound fields, and missing values. To address these bottlenecks, this project requires building a modular data pipeline capable of extracting massive amounts of data, cleaning it, and applying comparative optimization techniques to process it efficiently.

Objectives

The primary objectives of this project are:

- To extract a large-scale dataset of over 100,000 records from a dynamic web source using an automated headless browser framework.
- To design a data processing pipeline that cleans and transforms the raw, unstructured scraped data into a structured format ready for analysis.
- To evaluate the computational efficiency of data manipulation by establishing a standard, single-threaded Pandas baseline.
- To apply and benchmark at least two high-performance optimization techniques specifically Python Multiprocessing,Pandas and Polars DataFrame library.
- To profile and compare the execution time, peak memory usage, average CPU utilization, and throughput across the different processing methods.

Target website and Data To Be Extracted

The target data source for this project is carlist.my which is one of the largest online car marketplaces in Malaysia. Because the platform renders its national listings index dynamically using JavaScript, a plain HTTP request would return no usable data; therefore, Playwright was utilized to render the pages and extract the elements directly from the DOM. The crawler is designed to gather approximately 150,015 raw records. For each vehicle listing, the scraper targets 11 specific attributes. The core data points extracted from each listing card include the car's title,

manufacturing year, price, and mileage. Additionally, attributes such as the vehicle's condition such as USED, RECON, or NEW, are directly inferred from the listing URL.

2.0 System Design and Architecture

2.1 Description of Architecture

The system architecture illustrates the end-to-end data lifecycle of the WebMiner project, ensuring a scalable, modular pipeline designed to handle large-scale web scraping and data processing.

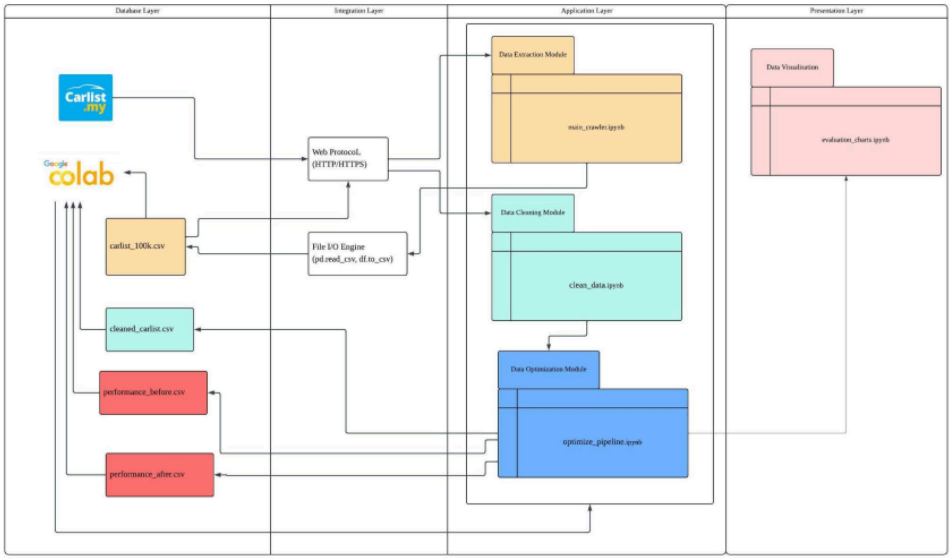

Figure 1: System Architecture

Based on the provided architectural flowchart, the system is organized into four distinct layers

Database Layer: This layer serves as the primary storage environment hosted within Google Colab. It houses the raw unstructured data (carlist_100k.csv), the intermediate sanitized data (cleaned_carlist.csv), and the final computational metrics (performance_before.csv, performance_after.csv).

Integration Layer: This layer bridges the external environment and internal storage. It utilizes Web Protocols (HTTP/HTTPS) to interface with Carlist.my and relies on File I/O Engines (pd.read_csv, df.to_csv) to transfer data smoothly between storage and application scripts.

Application Layer: The core logic engine of the project, divided into three sequential execution modules: the Data Extraction Module (main_crawler.ipynb), the Data Cleaning Module (clean_data.ipynb), and the Data Optimization Module (optimize_pipeline.ipynb).

Presentation Layer: The final phase (evaluation_charts.ipynb), which ingests the generated performance files to produce analytical Data Visualizations for comparative evaluation.

2.2 Tools and Frameworks Used

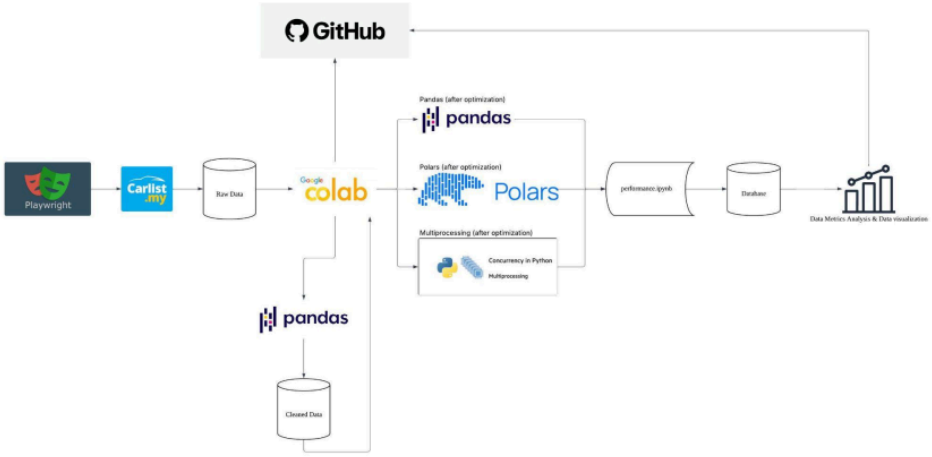

Figure 2: Architecture of Tools and Frameworks used

While the system architecture defines the logical data flow, the tools architecture highlights the specific technology stack chosen to execute each pipeline stage efficiently.

GitHub & Google Colab: GitHub acts as the overarching version control system, tracking code changes and syncing the repository. Google Colab serves as the central cloud execution environment where the raw data is stored and scripts are run.

Playwright: Used as the primary extraction tool to interact with Carlist.my, circumventing dynamic web elements and rendering JavaScript to capture the raw data accurately.

Pandas: Serves a dual purpose in the pipeline. It acts as the primary data manipulation framework to produce the cleaned baseline data, and it is also tested later as the single-threaded control variable during the optimization phase.

Polars: Deployed as the software-level optimization engine. Written in Rust, it leverages lazy evaluation and native multi-threading to process the data with significantly higher memory efficiency and speed than standard Pandas.

Multiprocessing: Utilized as a hardware-level optimization strategy to achieve concurrency in Python. It splits the processing tasks to fully utilize multiple CPU cores, bypassing the limitations of single-threaded execution.

Seaborn/Matplotlib: Integrated at the end of the pipeline to transform the performance.csv outputs into comprehensive data visualizations and metrics analysis.

2.3 Roles of Team Members

To successfully orchestrate this pipeline, responsibilities were divided among the WebMiner team members to cover extraction, transformation, and high-performance optimization:

- Muhammad Syahmi Faris bin Rusli (Data Extraction): Syahmi was responsible for the Data Extraction Module. He spearheaded the web crawling infrastructure, utilizing Playwright in main_crawler.ipynb to successfully navigate Carlist.my, bypass dynamic rendering roadblocks, and safely extract the raw automotive listings into the initial database layer.

- Ng Yu Hin (Data Cleaning): Yuhin was responsible for the Data Cleaning Module. He handled the foundational data sanitization using Pandas in clean_data.ipynb, transforming the messy, unstructured web text into a clean, standardized schema by executing deduplication, handling missing values, and formatting pricing and mileage strings.

- Muhammad Afiq Danish bin Mohd Hazni (Pipeline Optimization): Afiq was responsible for the Data Optimization Module. He implemented the high-performance computing frameworks in optimize_pipeline.ipynb by comparing Polars and Multiprocessing against Pandas. He was also responsible for profiling the hardware metrics and charting the comparative analysis in the Presentation Layer.

3.0 Data Collection

The dataset was collected from carlist.my (https://www.carlist.my/), one of Malaysia's largest online car marketplaces, by crawling its national listings index. A total of 150,015 raw records were gathered, each with 11 attributes.

Crawling Method. The crawler was built in Python using Playwright, a headless browser framework. A browser-based approach was necessary because carlist.my renders its listings via JavasScript, so a plain HTTP request would return no usable data. Key design features:

- **Asynchronous Concurrency:** uses asyncio with a semaphore limiting the crawler to 4 concurrent browser context, fetching multiple pages in parallel.
- **Pagination Traversal:** iterates through listing pages (25 per page) using page_number query parameters unti the target count is reached.
- **Direct DOM Extraction:** all 11 fields are extracted in asingle in-page page.evaluate() call, reading each article listing card for title, year, price, mileage and other attributes. Condition (USED/RECON/NEW) is inferred from the URL.
- **Resilience:** each request retries up to 3 times with exponential backoff, handling HTTP 403/429 rate limit responses, fonts, images, and analytics requests are blocked to speed up loading.
- **Resumability:** progress and seen IDs are persisted, so the crawler can resume after interruption and skip duplicates.

Ethical Considerations. Only publicly accessible listing data was collected, no login or personal data. Server load was kept modest through limited concurrency, a 1.5 second delay per page, and backoff on rate limiting. The data is used strictly for academic purposes and is not redistributed.

4.0 Data Processing

The raw dataset contained inconsistencies typical web-scraped data which is numbers stored as strings, noisy text fields, compound fields, missing values, and outliers. A cleaning pipeline was built in a Jupyter Notebook which is clean_data.ipynb using Pandas to produce the analysis ready carlist_cleaned.csv.

Raw Data Structure: The raw file has 150,015 rows x 11 columns. Issues included price and monthly_payment as currency strings, mileage as a range string, seller_type as noisy text, location combining state and city, and unrealistic year values. Missing values appeared in monthly_payment (187), seller_type (55), and id (1).

Cleaning Steps:

1. **Duplicate removal:** dropped exact duplicate rows
2. **Price parsing:** RM 63,999 changed into 63999.0, a filter (RM1,000 - RM5,000,000) removed 112 outliers
3. **Monthly payment parsing:** converted to numeric, around 187 missing values imputed with the median
4. **Mileage parsing:** range string converted to a numeric midpoint (75 - 80 KM to 77,500)
5. **Seller type extraction:** reduced noisy text to clean labels (Dealer, Sales Agent, Private, Broker and Other)
6. **Location splitting:** split into separate state and city columns
7. **Year filtering:** ke[t 1980 - 2026, removing 21 outliers
8. **Categorical standardisation:** condition and transmission trimmed and Title-cased
9. **Schema finalisation:** dropped redundant raw columns, leaving a 12-column schema

Final Data Structure: After removing 133 records in total, the final dataset contains 159,882 rows x 12 columns: id, title, year, price_myr, monthly_payment_myr, mileage_km, transmission, seller_type, state, city, condition, and url. This cleaned dataset services as the common input fot all three optimisation techniques in Section 5.

5.0 Optimization Techniques

This section describes the two optimization techniques applied to the data processing pipeline alongside the baseline approach used for comparison. All three methods execute a similar analytical pipeline on the same dataset which is carlist_cleaned.csv containing 149,882 records and 12 features so that performance differences can be attributed solely to the technique being used.

The process benchmarked in each method consists of five stages:

1. Load the CSV file.
2. Filter rows to select only used cars with mileage below 100,000 km.
3. Feature engineering process which computes three derived features (*price_per_km, car_age, price_per_year*).
4. Aggregations by state and transmission type.
5. Finding top 10 best value cars

5.1 Baseline Pandas

The baseline uses the standard Pandas library with no parallelism. All operations execute sequentially on a single CPU core which is the most common approach in typical Python workflows.

The pipeline loads the dataset using *pd.read_csv()*, applies boolean conditioning for row filtering, and creates new columns through direct arithmetic. Groupby aggregations are then performed using *.groupby().agg()*. The top 10 best value cars are identified using *nsmallest(10, 'price_per_km')*, which selects the 10 rows with the lowest price-to-mileage ratio. It represents cars that offer the most distance covered per ringgit spent. Because Pandas processes all of these steps sequentially in a single thread, execution is bounded by the speed of one core and the overhead of Python's garbage-collected memory model.

```python
def baseline_analysis(csv_path):
    # ── Step 1: Load data ──────────────────────────
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # ── Step 2: Filter ─────────────────────────────
    df_f = df[
        (df['mileage_km'] < 100_000) &
        (df['condition'] == 'Used') &
        (df['price_myr'] > 0)
    ].copy()

    # ── Step 3: Feature Engineering ────────────────
    df_f['price_per_km']   = df_f['price_myr'] / (df_f['mileage_km'] + 1)
    df_f['car_age']        = CURRENT_YEAR - df_f['year']
    df_f['price_per_year'] = df_f['price_myr'] / (df_f['car_age'] + 1)

    # ── Step 4: Aggregate by state ─────────────────
    state_stats = df_f.groupby('state').agg(
        avg_price    = ('price_myr',  'mean'),
        car_count    = ('id',         'count'),
        avg_mileage  = ('mileage_km', 'mean'),
        median_price = ('price_myr',  'median')
    ).reset_index().sort_values('avg_price', ascending=False)

    # ── Step 5: Aggregate by transmission ──────────
    trans_stats = df_f.groupby('transmission').agg(
        avg_price   = ('price_myr',  'mean'),
        car_count   = ('id',         'count'),
        avg_mileage = ('mileage_km', 'mean')
    ).reset_index()

    # ── Step 6: Top 10 best value cars ─────────────
    top_value = df_f.nsmallest(10, 'price_per_km')[
        ['title', 'state', 'price_myr', 'mileage_km', 'price_per_km']
    ]

    return total_rows, len(df_f), state_stats, trans_stats, top_value
```

Figure 3: Code Snippet of Pipeline Execution Using Pandas

This baseline establishes the reference performance figures against which both optimization techniques are measured.

5.2 Multiprocessing

The first optimization technique used is Python's multiprocessing module. The idea behind this technique is to split the dataset into smaller pieces and process each piece at the same time using different CPU cores. Since the machine used has 16 CPU cores, the dataset was split into 16 chunks and each chunk was sent to a different worker process.

The reason multiprocessing was chosen instead of multithreading is because of the Global Interpreter Lock, or GIL. The GIL is a limitation in Python that stops multiple threads from running Python code at the same time. This means that even when multiple threads are used, it cannot actually run in parallel for CPU-heavy tasks like calculations. Multiprocessing gets around this problem by using separate processes instead of threads, so each process has its own Python interpreter and there is no GIL limitation.

One issue that was encountered is that the worker functions cannot be defined inside the Jupyter Notebook. This is because on Windows, Python creates new processes by re-importing the main script, and when the main script is a Jupyter Notebook, this causes the program to hang and never finish.

```python
def process_chunk(args):
    """
    Runs in each worker process.
    Applies filter + feature engineering on one DataFrame chunk.
    """
    chunk_df, _ = args

    filtered = chunk_df[
        (chunk_df['mileage_km'] < 100_000) &
        (chunk_df['condition'] == 'Used') &
        (chunk_df['price_myr'] > 0)
    ].copy()

    filtered['price_per_km']   = filtered['price_myr'] / (filtered['mileage_km'] + 1)
    filtered['car_age']        = CURRENT_YEAR - filtered['year']
    filtered['price_per_year'] = filtered['price_myr'] / (filtered['car_age'] + 1)

    return filtered
```

Figure 4: Code Snippet of *mp_helpers.py*

To fix this, the worker function was moved into a separate file called *mp_helpers.py*. The *joblib* library with the *loky* backend was also used to manage the worker processes, because it handles this situation better than using *multiprocessing.Pool* directly.

```python
def multiprocessing_analysis(csv_path, n_workers):
    # ── Step 1: Load data (serial – file I/O) ──────────
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # ── Step 2: Split DataFrame into N chunks (pandas iloc) –
    chunks = chunk_dataframe(df, n_workers)
    args   = [(chunk, i) for i, chunk in enumerate(chunks)]

    # ── Step 3: Process chunks in parallel ─────────────
    # joblib loky backend avoids the Windows+Jupyter spawn hang
    results = Parallel(n_jobs=n_workers, backend='loky')(
        delayed(process_chunk)(arg) for arg in args
    )

    # ── Step 4: Merge results from all workers ─────────
    df_f = pd.concat(results, ignore_index=True)
```

Figure 5: Code Snippet of Pipeline Execution Using Multiprocessing

After all the worker processes finish, the results are combined back together using *pd.concat()* in the main process. Then, the aggregation steps are run on the combined data.

5.3 Polars

The second optimization technique used is Polars which is a DataFrame library written in a programming language called Rust. Polars works differently from Pandas because it does not run each operation immediately. Instead, it first builds a plan of all the steps that need to be done, then optimizes that plan and only then actually runs everything at once. This is called lazy evaluation.

Polars is used by calling *pl.scan_csv()* instead of reading the file directly. This does not load the file yet. After that, filter and feature engineering steps are added to the plan. When *.collect()* is finally called, Polars looks at the full plan, figures out the most efficient way to run it and then executes everything using multiple CPU cores automatically. There is no need to manually split the data or create worker processes like in the multiprocessing approach.

Polars also stores data in a format called Apache Arrow which organizes data by columns instead of by rows. This makes operations like filtering and calculating averages faster because the CPU only needs to read the relevant columns instead of going through every row.

```python
def polars_analysis(csv_path):
    # ── Step 1: Lazy scan — file is NOT loaded yet ──────────
    # Polars builds a query plan and optimizes before executing
    lf = pl.scan_csv(csv_path)

    # ── Step 2: Filter (lazy) ────────────────────────────────
    lf_f = lf.filter(
        (pl.col('mileage_km') < 100_000) &
        (pl.col('condition') == 'Used') &
        (pl.col('price_myr') > 0)
    )

    # ── Step 3: Feature engineering (lazy) ──────────────────
    lf_f = lf_f.with_columns([
        (pl.col('price_myr') / (pl.col('mileage_km') + 1)).alias('price_per_km'),
        (CURRENT_YEAR - pl.col('year')).alias('car_age'),
        (pl.col('price_myr') / (CURRENT_YEAR - pl.col('year') + 1)).alias('price_per_year')
    ])

    # ── Step 4: Collect — executes the full optimized plan ──
    df_f = lf_f.collect()
    total_rows = pl.scan_csv(csv_path).select(pl.len()).collect().item()
    filtered_rows = len(df_f)
```

Figure 6: Code Snippet of Pipeline Execution Using Polars

Compared to multiprocessing, Polars is easier to use because there is no need to manually manage processes or worry about how data is passed between workers. The parallelism happens inside Polars itself without any extra setup required.

6.0 Performance Evaluation

6.1 Performance Measurement Function

To measure the performance of each method, a tool called *MetricsCollector* was built. It is a context manager which means it can be wrapped around any block of code and it will automatically record the performance while that code is running. Four metrics were measured for each method:

- Execution time is how long the code takes to finish, measured using *time.perf_counter()* which is a very accurate timer.
- Peak memory is the highest amount of memory used during the run. This was measured using two tools together, *tracemalloc* which tracks memory used by Python and *psutil* which also tracks memory used by non-Python parts like Rust or C libraries.
- Average CPU utilization is how much of the CPU was being used on average. This was checked every 0.1 seconds using *psutil.cpu_percent()* running in a background and then the average was calculated.
- Throughput is how many rows the method can process per second, calculated by dividing the total number of rows by the time taken.

Each method was also run five times and the average of the results was taken. This was done because the first run is sometimes slower due to the computer needing to load the file into memory for the first time. By averaging five runs, a more accurate and fair result is obtained.

6.2 Results

The table below shows the average performance results for all three methods across five runs.

Metric	Pandas	Multiprocessing	Polars
Execution Time (s)	0.631	0.906	0.059
Peak Memory (MB)	66.8	99.6	33.8
Average CPU Utilization (%)	25.4	23.3	30.7
Throughput (rows/s)	237753	165629	2547055

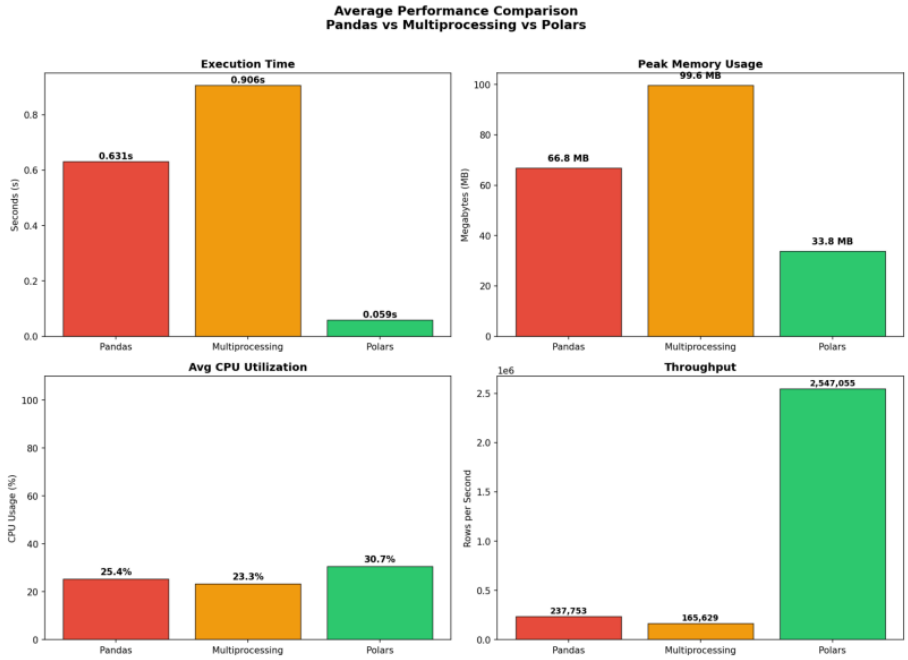

Figure 7: Performance Comparison Charts

Figure 7 shows the performance comparison charts for all three methods across the four metrics. Looking at the execution time chart, Polars is the fastest method at 0.059 seconds while Multiprocessing is the slowest at 0.906 seconds, which is even higher than the Pandas baseline at 0.631 seconds. This shows that having more CPU cores does not always mean the program will run faster.

For peak memory usage, Multiprocessing used the most memory at 99.6 MB which is almost three times more than Polars at 33.8 MB. Pandas falls in the middle at 66.8 MB. The high memory usage of Multiprocessing is because copies of the data need to be made for each of the 16 worker processes.

The CPU utilization chart shows that all three methods have similar CPU usage levels. Polars had the highest at 30.7%, followed by Pandas at 25.4%, and Multiprocessing at 23.3%. Even though Multiprocessing uses 16 worker processes, its average CPU usage is actually the lowest, which suggests that the workers were spending more time waiting for data than actually doing computation.

The throughput chart shows the biggest difference between the methods. Polars processed approximately 2,547,055 rows per second which is about 10.7 times more than Pandas at 237,753 rows per second, and about 15.4 times more than Multiprocessing at 165,629 rows per second. This clearly shows that Polars is the most efficient method for processing this dataset.

6.3 Results Analysis

Execution Time: Polars finished the pipeline in 0.059 seconds on average which is about 10.7 times faster than Pandas at 0.631 seconds. Multiprocessing took 0.906 seconds which is actually 43.6% slower than Pandas even though it was using all 16 CPU cores. This happened because the dataset only has about 149,882 rows as it is relatively small. Before any actual processing can happen, Multiprocessing needs to split the data into 16 chunks, send each chunk to a separate worker process and then combine all the results back together at the end. For a dataset of this size, the time spent on these extra steps is more than the time saved by running things in parallel.

Memory Usage: Polars used the least memory at 33.8 MB which is about 49.4% less than Pandas at 66.8 MB and about 66% less than Multiprocessing at 99.6 MB. The reason Polars uses less memory is because of its lazy evaluation approach since it filters out unwanted rows before fully loading the data, there is less data that needs to be stored in memory at any one time. Multiprocessing uses the most memory because the dataset has to be copied and sent to each of the 16 worker processes separately that results in many copies of the data being stored in memory at the same time.

CPU Utilization: The average CPU usage was fairly similar across all three methods, ranging from 23.3% to 30.7%. Polars had the highest at 30.7%, which shows that it was making use of multiple CPU cores through its internal thread pool. Multiprocessing had the lowest at 23.3%, which is surprising for a method that uses 16 worker processes. This suggests that most of the time was spent on coordinating between processes rather than on doing actual computation.

Throughput: Polars processed about 2.55 million rows per second compared to 237,753 rows per second for Pandas and 165,629 rows per second for Multiprocessing. This means Polars is about 10.7 times faster than Pandas and about 15.4 times faster than Multiprocessing in terms of data processed per second. This is the clearest indicator of how much more efficient Polars is compared to the other two methods.

7.0 Challenges and Limitations

7.1 Multiprocessing Not Working in Jupyter on Windows

One of the biggest problems encountered was that multiprocessing did not work properly when run inside a Jupyter Notebook on Windows. The program would just hang and never finish running. This happens because of how Windows creates new processes. On Windows, Python creates worker processes by re-importing the main script. Since the main script in Jupyter is the notebook kernel, the worker processes try to start the kernel again, which causes everything to get stuck.

7.2 Multiprocessing Was Slower Than the Baseline

Multiprocessing turned out to be slower than the Pandas baseline even though it used 16 CPU cores. This was not an expected result. The problem is that the dataset is too small for multiprocessing to be useful. Before any actual processing can happen, the program needs to split the data into 16 pieces and send each piece to a worker process through a mechanism called inter-process communication (IPC). Then, it combines all the results back together at the end. For a dataset of 149,882 rows, the time spent on these extra steps is more than the time saved by running things in parallel. Multiprocessing would likely give better results if the dataset was much larger such as several million rows.

7.3 Limitations of the CPU Measurement

The way CPU usage was measured has some limitations. The measurement is taken across the whole system, not just for the program being tested. This means that if other programs on the computer were also using the CPU during the benchmark, those numbers would be included in the readings. Also, because the sampling is done every 0.1 seconds, short bursts of high CPU activity that happen in between samples may be missed. These limitations mean that the CPU numbers reported should be treated as estimates rather than exact values, though they are still useful for comparing the three methods against each other.

8.0 Conclusion

8.1 Summary of Findings

The WebMiner project successfully architected and executed an end-to-end Big Data pipeline, transforming raw, unstructured web data into an optimized, analysis-ready dataset. By successfully scraping and cleaning approximately 100,000 automotive records from Carlist.my, the project established a rigorous testing ground to evaluate the efficiency of different data processing paradigms.

The comparative performance evaluation yielded definitive conclusions regarding high-performance computing on medium-to-large tabular datasets:

- **Software-Level Optimization Dominates: Polars** proved to be vastly superior, completing the pipeline execution in just 0.059 seconds and processing roughly 2.55 million rows per second. It operated **10.7 times faster** than the Pandas baseline and **15.4 times faster** than Multiprocessing.
- **Superior Memory Management:** Polars was also the most memory-efficient, consuming only 33.8 MB at its peak. This is largely due to its lazy evaluation query engine, which filters unnecessary data before loading it into memory, consuming roughly half the RAM of Pandas (66.8 MB).
- **The Overhead Trap of Multiprocessing:** Python's native **Multiprocessing** was the least efficient approach (0.906 seconds execution time, 99.6 MB peak memory). This finding highlights a critical Big Data principle: hardware-level concurrency introduces severe overhead. The computational cost of copying the dataset, distributing it across 16 CPU cores, and concatenating the results back together completely negated the benefits of parallel execution for a dataset of this size.

Ultimately, this project validates that for modern data engineering workloads, leveraging software-level optimizations such as rust-based execution and lazy evaluation, is significantly more efficient than relying on brute-force hardware scaling.

8.2 What Could Be Improved

While the current architecture successfully met the project's objectives, scaling this system to an enterprise-level production environment would require several architectural upgrades.

- **API-Level Data Extraction:** Currently, the pipeline relies on Playwright to render JavaScript and scrape the Document Object Model (DOM). A major improvement would involve reverse-engineering the site's network requests to locate the hidden JSON APIs. Extracting data directly via standard HTTP requests would bypass browser rendering entirely, reducing extraction time from hours to mere minutes.

- **Distributed Computing for TB-Scale Data:** While Polars is exceptional for out-of-core processing on a single machine, it is bound by the hardware limits of Google Colab. If the project were to scale to millions of records across multiple international property/automotive sites, transitioning to a distributed computing framework like **Apache Spark (PySpark)** or **Dask** would be necessary to distribute the workload across a cluster of machines.

- **Automated Pipeline Orchestration:** The current pipeline relies on the manual execution of sequential Jupyter Notebooks which starts from main_crawler.ipynb to clean_data.ipynb to optimize_pipeline.ipynb. Integrating an orchestration tool like **Apache Airflow** or **Mage.ai** would allow the pipeline to run autonomously on a scheduled basis featuring automated retries and error alerting.

- **Cloud Data Warehousing:** Storing the processed records in flat CSV files is sufficient for benchmarking, but inefficient for long-term querying. Future iterations should connect the output layer directly to a structured relational database such as PostgreSQL or a cloud data warehouse such as Google BigQuery or Amazon Redshift to facilitate seamless business intelligence dashboarding.

Final_Report (WebMiner).pdf

ORIGINALITY REPORT

3%	0%	0%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Universiti Teknologi Malaysia Student Paper	2%
2	Submitted to Organisation et Développement Student Paper	1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		